Go through the below topics with concepts and real-time examples.

SQL EXECUTION ORDER

Stages of SQL execution order

Here is the typical order of execution in SQL:

- FROM and/or JOIN clause
- WHERE clause
- GROUP BY clause
- HAVING clause
- SELECT statement
- DISTINCT clause
- ORDER BY clause
- LIMIT and/or OFFSET clause

Sample SQL execution order with example

id	name	department	salary	active
1	Alice	Sales	60000	1
2	Bob	Engineering	80000	1
3	Carol	Sales	62000	1
4	Dave	HR	50000	0
5	Eve	Engineering	82000	1
6	Frank	Sales	58000	1
7	Grace	Engineering	79000	1
8	Heidi	Sales	61000	1

```
SELECT department, COUNT(*) AS employee_count
FROM employees
WHERE active = 1
GROUP BY department
HAVING COUNT(*) > 2
ORDER BY employee_count DESC
```

Let's analyze how this query is processed step-by-step.

Step 1: The FROM clause loads all the data from the employees table.

Step 2: The WHERE clause filters the rows where active = 1, keeping only active employees.

Step 3: The GROUP BY clause groups the filtered rows by the department column.

Step 4: The HAVING clause keeps only those departments with more than 2 employees.

Step 5: The SELECT clause selects the department name and the count of employees in each retained group.

Step 6: The ORDER BY clause sorts the result set by employee_count in descending order.

Step 7: The LIMIT clause returns only the first two rows from the sorted result set.

Here is the output for query:

department	employee_count
Sales	4
Engineering	3

SQL Clauses:

1. Select& From -Used to retrieve specific columns from a table.

2. Where -Used to filter rows before grouping.

3.Group by -Groups rows with similar values.

4.Having -Filters grouped data (used after GROUP BY).

5.Order by -Used to sort results.

Aggregate Functions:

1.MIN() -returns the smallest value of a column.

2. MAX() - returns the largest value of a column

3. COUNT() - returns the number of rows in a set

4.SUM() - returns the sum of a numerical column

5.AVG() - returns the average value of a numerical column

Window functions:

1.ROW_NUMBER() –Assigns a **unique sequential number** to each row within a partition

2.RANK() –Assigns a **rank to rows based on order**, but **skips rank numbers when there are ties**.

3.DENSE_RANK() –Assigns a **rank without skipping numbers**, even if there are ties.

4.SUM() OVER() –Calculates the **running total (cumulative sum)** across rows without grouping them.

5.AVG() OVER() –Calculates the **running average** of values across rows.

6.MIN() OVER() –Returns the **maximum value within the window or partition** while keeping all rows.

7.MAX() OVER() -Returns the **minimum value within the window or partition** while keeping all rows.

SQL Joins:

1.**Inner join**-Returns only the rows that have matching values in both tables.

2.**Left join**-Returns all rows from the left table and matched rows from the right table, otherwise NULL.

3**.Right join**-Returns all rows from the right table and matched rows from the left table, otherwise NULL.

4.**Full join**-Returns all rows from both tables, matching where possible and filling NULL where no match exists

5.**Self join**-Joins a table with itself to compare rows within the same table.

6.**Cross join** -Returns the Cartesian product (all possible combinations) of rows from both tables.

7.**Left join(with NULL check)-** Used to find rows in the left table that do not have matching rows in the right table.

8.**Right join (with NULL check)-** Used to find rows in the right table that do not have matching rows in the left table.

Keys in Relational Model

1.Candidate Key-The minimal set of attributes that can uniquely identify a tuple

2.Super Key-The set of one or more attributes (columns) that can uniquely identify a tuple (record)

3.Alternate Key-all the keys that are not selected as the primary key are considered alternate keys.

4.Foreign Key-an attribute in one table that refers to the primary key in another table

5.Primary Key-chosen from the set of candidate keys to uniquely identify each record in a table

6.Partial Key

7.Secondary Key

8.Unique Key

9.Composite Key

10.Surrogate Key

SQL Clauses

1. What is the purpose of SQL clauses in a query?
2. Why is the WHERE clause used in SQL queries?
3. When should the HAVING clause be used instead of WHERE?
4. What is the role of the GROUP BY clause in SQL?
5. Why is ORDER BY used in SQL queries?
6. How does the DISTINCT clause help in retrieving data?
7. What happens if GROUP BY is used without an aggregate function?
8. Why is LIMIT or TOP used in SQL queries?
9. How does SQL handle filtering before and after grouping?
10. When would you use multiple clauses together in a single query?

SQL Query Execution Order

1. What is the logical execution order of a SQL query?
2. Why does SQL execute the FROM clause before the SELECT clause?
3. Why can't column aliases be used in the WHERE clause?
4. When does SQL apply the WHERE condition in the query execution process?
5. How does SQL process JOIN operations during query execution?
6. Why are aggregate functions evaluated after GROUP BY?
7. When is the HAVING clause executed during query processing?
8. How does SQL handle sorting of data during query execution?
9. Why are window functions executed after aggregation?
10. What would happen if SQL did not follow a fixed execution order?

SQL Joins

1. What is the purpose of joins in relational databases?
2. Why are joins required when working with multiple tables?
3. When should INNER JOIN be used?
4. When is LEFT JOIN preferred over INNER JOIN?
5. What is the difference between LEFT JOIN and RIGHT JOIN?

6. How does a FULL JOIN combine records from two tables?
7. What is a SELF JOIN and why is it useful?
8. What is the purpose of CROSS JOIN?
9. How can joins help in maintaining relationships between tables?
10. When would you check for NULL values after a LEFT or RIGHT JOIN?

Keys in Relational Model

1. What is a key in the relational database model?
2. Why is a primary key important for a table?
3. How does a foreign key maintain referential integrity?
4. What is the difference between a candidate key and a primary key?
5. Why are composite keys used in database tables?
6. What is a super key and how is it different from other keys?
7. When should a surrogate key be used instead of a natural key?
8. Why can't primary keys contain NULL values?
9. How do keys help in defining relationships between tables?
10. What problems might occur if a table does not have a primary key?

Aggregate Functions

1. What are aggregate functions in SQL?
2. Why are aggregate functions used in data analysis?
3. When should GROUP BY be used with aggregate functions?
4. What is the difference between COUNT(*) and COUNT(column)?
5. Why can't aggregate functions be used in the WHERE clause?
6. How does SQL calculate average values using AVG()?
7. What happens when aggregate functions are used without grouping?
8. Why is HAVING used with aggregate functions?
9. How do aggregate functions help summarize large datasets?
10. When should multiple aggregate functions be used in a query?

Window Functions

1. What are window functions in SQL?
2. How do window functions differ from aggregate functions?
3. Why is the OVER() clause required in window functions?
4. What is the purpose of PARTITION BY in window functions?
5. When should ROW_NUMBER() be used?
6. What is the difference between RANK() and DENSE_RANK()?
7. How do window functions help in ranking data?
8. When should window functions be used instead of GROUP BY?
9. How do running totals work in window functions?
10. Why are window functions useful for analytical queries?